IS469 Topic 4 Notes: Customising LLMs

Oct 25, 2026 · @yanting

1. The customisation ladder

Fine-tuning is the last rung, not the first. Climb only as far as your evals force you to; each rung costs more and is harder to reverse.

1. **Evals first** — measure before you change anything.
2. **Prompting & context** — instructions, examples, structure. Cheapest and reversible.
3. **Fine-tune** — only when a behaviour gap remains.
4. **Fine-tune efficiently** — LoRA / QLoRA on one GPU (this topic). Full fine-tuning is rarely necessary now.

Knowledge gap vs behaviour gap

First diagnose what is actually failing — the fix depends on the gap.

Gap	Symptom	Fix
Knowledge	Missing, stale or private facts	Not fine-tuning (use context / retrieval)
Behaviour	Wrong format, tone, policy, unreliable reasoning	Fine-tune
Own label set	Classification, ranking	Train for it directly

- Volatile knowledge (prices, regulations, docs) should never be fine-tuned in — it goes stale.
- Fine-tuning teaches a model **how to behave**, far more than **what to know**.

Where fine-tuning still clearly wins

- Strict output structure: JSON / schema adherence, house style, persona.
- Latency & cost: distil a task into a 1–8B model instead of calling a frontier API.
- Verifiable skills: math, code, structured extraction, reliable tool-calling.
- Domain encoders (classification, retrieval, ranking) almost always need it.
- Private data that cannot leave your infrastructure.

How much data?

- No single number — it scales with what the model has to learn.
- Style, tone or format: a few hundred clean examples often suffice (accuracy plateaus around 200–500).
- Classification is **not** a behaviour task: budget **per class**, ~50–100 per class (5 classes ≈ a few hundred; 100 classes ≈ thousands).
- The learning curve on your own data says when to stop, not a rule of thumb.

2. The efficiency problem & training libraries

Once you decide to fine-tune, **GPU memory, not compute, is the wall**. Full fine-tuning of even a few-billion-param model overruns a single consumer GPU.

- 2026 open-weight families ship in many sizes: Qwen3, Llama-4, Gemma-3, DeepSeek, Phi-4. MoE is now the default frontier open architecture.

Full fine-tuning vs efficient customisation

	Full fine-tuning (FFT)	Efficient customisation
What trains	Every parameter	Tiny fraction (PEFT / LoRA)
Memory	Optimiser states dominate (~16 bytes/param)	Cut via mixed precision + quantisation
Hardware	Multi-GPU / datacentre for large models	Multi-billion-param model on one GPU
Output	One full weight copy per task	Small, shareable adapters

Main libraries

- **Hugging Face** — default high-level training stack.
- **PyTorch FSDP** (Fully Sharded Data Parallel) — distributed training across many GPUs.
- **DeepSpeed** (Microsoft) — distributed training + inference; actively maintained.
- In practice you compose them: HF on top, FSDP or DeepSpeed underneath for scale.

The Hugging Face ecosystem

Library	What it does
Transformers	Load & train models — the core library
Accelerate	Thin layer for distributed / mixed-precision training
PEFT	Parameter-efficient fine-tuning (LoRA and friends)
TRL	Post-training: SFT, preference (DPO), reinforcement (GRPO) — one Trainer per method
Hub	Hosts most open-weight models (audio & vision too)

Trainer & SFTTrainer

- `pipeline()` — quick out-of-the-box inference; not for serious training.
- `Trainer` — wrapper over the PyTorch training loop; sets hyperparameters cleanly.
- `SFTTrainer` (TRL) — wrapper over Trainer specialised for SFT. Handles model + dataset loading, tokenising, batching, data collator and training args. The default used in the lab.

3. Supervised fine-tuning (SFT)

SFT is the workhorse: show the model labelled prompt → answer pairs, and score **only the answer tokens**.

1. **Prompt given** — the question.
2. **Model predicts** — the completion / answer.
3. **Loss** — cross-entropy on the answer tokens only; prompt tokens are masked.

- In TRL this is the `train_on_completions_only` behaviour.
- API note (2026): SFT args like `dataset_text_field` and `max_seq_length` now live in `SFTConfig`, not `TrainingArguments`.

What SFT cannot do

- SFT imitates demonstrations — it copies the answers you show it.
- It has no notion that one answer is *better* than another, only "this is the target".
- So it can't directly optimise for helpfulness, safety or a preferred style → that needs preference / RL methods (next section).

4. Preference & reinforcement fine-tuning

The 2025–26 shift: optimise for **better** answers, not just imitate given ones. Pipeline: pre-train → SFT → preference / RL alignment. All of it composes with LoRA / QLoRA.

- **Preference tuning** learns from comparisons: answer A is better than answer B.
- **Reinforcement fine-tuning (RFT)** rewards good behaviour and penalises bad, over many tries.

Method	Learns from	Needs a reward model?
SFT	Labelled prompt → answer pairs	No
DPO	Preference pairs (chosen vs rejected)	No — closed-form update
RLHF (PPO)	Reward model trained on preferences	Yes
GRPO / RFT	Reward signal per sampled answer	Often no — rule-based reward

Trend: away from a separate reward model, toward direct (DPO) or rule-based / verifiable (GRPO) rewards.

DPO (Direct Preference Optimization)

- Collect pairs per prompt: a **chosen** and a **rejected** answer.
- DPO derives a closed-form loss straight from the pairs — no RL loop, no reward model.
- Simpler and cheaper than RLHF; **offline** (no sampling during training).
- Used to post-train Llama 3 and many other open models. In HF: TRL's `DPOTrainer`.

RFT / RLVR (verifiable rewards)

- Reward is a **checkable rule**, not a learned model — e.g. is the math/code/extraction/tool-call correct?
- Model samples answers; correct ones are reinforced → it learns the behaviour, not one label.
- **GRPO** (DeepSeekMath; popularised by DeepSeek-R1, early 2025) is the workhorse algorithm.
- Payoff: a 7–13B model tuned this way can rival a 70B frontier model on its narrow task.

5. Stable training & distributed training

Gradient accumulation gives bigger effective batches on small GPUs; weight decay and clipping keep gradients under control.

Gradient accumulation

1. Run forward + backward on batches 1…k.
2. Sum the gradients — don't update yet.
3. Step the optimiser once per k batches.

```
\text{effective batch} = \text{per-iter batch} \times \text{accumulation steps} \times \text{num procs}
```

- Bigger effective batch → less noisy gradient → scale the learning rate **up** to match.
- Rules of thumb: scale LR linearly with effective batch, or by its square root with Adam / AdamW.
- Watch convergence; reduce accumulation steps if training is too slow or erratic.
- Not free: gradients are stored until the update, which uses memory.

Exploding gradients

- A large spike in gradient norm → unstable training, can miss the optimum.
- **Weight decay (L2 regularisation)** — shrinks weights overall, reducing gradients.
- **Gradient clipping** — cap the gradient norm at a threshold during back-prop, then update. Standard for LLM fine-tuning.

Distributed training (recognise by name)

Paradigm	What it splits
Data parallelism (DDP)	Model replicated on each GPU; data split
FSDP / DeepSpeed ZeRO	Parameters, gradients and optimiser states sharded across GPUs
Pipeline parallelism	Layers split across GPUs
Tensor parallelism	Computation within a layer split across GPUs

Internals are graduate-level; just know these exist and why.

6. PEFT & LoRA

You don't need to move every weight: LLMs are heavily over-parameterised, so training a tiny low-rank update gets most of the benefit.

- **LoRA** — "Low-Rank Adaptation of Large Language Models" (Microsoft Research, 2021).
- Later methods haven't displaced it, except **QLoRA** (2023). **DoRA** (2024) applies LoRA to each weight's direction — modest uptake.
- Payoff: fine-tune much larger models on a single GPU.

How LoRA works

Full fine-tuning learns a ΔW the same shape as W_0. LoRA factorises it into two thin matrices and trains only those:

$$h = (W_0 + \Delta W)\,x = (W_0 + BA)\,x, \quad B \in \mathbb{R}^{d \times r},\; A \in \mathbb{R}^{r \times d},\; r \ll d$$

- W_0 is **frozen**; only A and B train. Folding BA back into W_0 is called **merging**.
- Example: d = 4096, r = 16 → W_0 = 16.8M params, A + B = 0.13M params ≈ **0.8%** trained (per layer).
- Analogy: instead of repainting the whole wall, you stick on a thin transparent overlay that shifts the picture.

Why two small matrices instead of one dense ΔW?

- **Memory** — only the d×r and r×d shapes (and their optimiser states) are stored.
- **Hardware** — Tensor Cores are built for dense matmul, which BA is.
- **Expressiveness** — ΔW is still dense, so every entry can move, but only within a low-rank subspace.

Why it works: (1) networks are over-parameterised, so training a small fraction still gets very good results; (2) by not touching the base weights, old knowledge isn't destroyed.

Adapters in practice

- Base weights frozen, never modified; only A, B trained → a small set of **adapters**.
- Typically ~0.01% of original params trained (you choose which layers).
- Adapters stored separately from base weights — pluggable and shareable.
- In HF / unsloth the model is patched in place → **reload the base each run, or adapters stack**.
- Number of steps per epoch = number of examples ÷ batch size.

LoRA hyperparameters (the knobs you tune in the lab)

Hyperparameter	Meaning	Typical
`r`	Rank of the update matrices; lower r = fewer trainable params (higher r = more params, longer training)	16
`target_modules`	Which layers get LoRA matrices (`"all-linear"` = every linear layer)	`"all-linear"`
`lora_alpha`	Scaling factor; update is (α / r)·(BA)x	α = r

7. Mixed precision & memory

Mixed precision combines FP32 and FP16/BF16 to cut memory while keeping quality; in full fine-tuning, optimiser states are the biggest memory cost.

The memory picture

- Industrial GPUs (A100): 40 or 80 GB. Consumer GPUs: often 24 GB.
- GPT-2 XL (1.5B params): ~6 GB weights in FP32, ~3 GB in FP16. GPT-2 small (124M): ~0.5 GB in FP32.
- Training needs far more than weights: optimiser states, gradients, activations, buffers.

Where the memory goes (N params, FP16 training with AdamW)

Component	Bytes / param
Params (FP16)	2
Gradients (FP16)	2
FP32 master param copy	4
AdamW momentum	4
AdamW variance	4
Total	**≈ 16**

- Example: 1.5B model ≈ 24 GB **before activations**.
- Activations add more — they scale with layers, sequence length, batch size, hidden size and heads.

- **FFT vs LoRA (~1.5B model):** params ~3 GB both; gradients 3 GB vs ~0; optimiser states 18 GB vs ~0. LoRA freezes the base, so its gradients and optimiser states nearly vanish.

AMP, FP16 vs BF16

- **AMP** (automatic mixed precision) matches each op to its best datatype: matmuls fine in low precision; norms and exp need high precision.
- **Dynamic loss scaling** shifts gradients into the representable range to avoid over/underflow.

Format	Sign	Exponent	Mantissa	Range	Note
FP32	1	8	23	~1e-38 to ~3e38	Full precision
BF16	1	8	7	~1e-38 to ~3e38	FP32's range, less precision → far fewer NaNs/Infs; = top 2 bytes of FP32
FP16	1	5	10	~5.9e-8 to 6.5e4	More precision, much narrower range → can over/underflow

8. Quantisation

Quantisation stores numbers in fewer bits → less memory and compute, at the cost of rounding error. This topic covers the training side (e.g. QLoRA); inference formats come in Topic 5.

- Low-precision types: int8, int4, fp8.
- Fewer bits = coarser buckets: FP32 ≈ 4.3B levels (4 bytes/weight), INT8 = 256 levels (1 byte), INT4 = 16 levels (0.5 byte). Each value rounds further from its true position.

int8: symmetric vs asymmetric

- int8 range is [-127, 127]; real values must be projected into it.
- **Symmetric** — scale by the largest absolute value → linear map centred at zero.
- **Asymmetric** — use actual min and max as the range (adds a zero-point offset).

```
X_Q = \operatorname{round}\!\left(X \cdot \frac{127}{\max|X|}\right) \qquad
X_D = \frac{\max|X|}{127} \cdot X_Q
```

De-quantisation

- Some ops must run in higher precision → de-quantise int8 back to FP16 / BF16 / FP32.
- Storing scale + zero-point makes de-quant far less lossy than naive.

- bitsandbytes (int8 / int4) stores extra tensors, `weight` and `SCB`, to de-quantise.
- Rounding loss is permanent — it can't be recovered on de-quant.

fp8: e4m3 and e5m2

- Arrived with newer GPUs (NVIDIA H100).
- **e4m3** — more precision → forward activations & weights.
- **e5m2** — more range (essentially truncated FP16) → gradients.
- fp8 range fits any single tensor but not all at once → **per-tensor scaling factors**.
- DeepSeek 3.1 introduced an fp8 ue8m0 scaling variant (for new Chinese GPUs).

Number formats at a glance

Format	Bits	Use	Note
FP32	32	Master weights / optimiser	Full-precision baseline
FP16	16	Mixed-precision compute	Narrow range → can over/underflow
BF16	16	Mixed-precision compute	FP32 range, fewer mantissa bits
int8	8	Quantised weights	Symmetric or asymmetric scaling
nf4	4	QLoRA frozen base	"Normal float", tuned for weights
MXFP4	4	Microscaling FP4 (OCP)	Block of 32 shares one scale; E2M1
NVFP4	4	Blackwell training	NVIDIA 4-bit float format

9. QLoRA, distillation & wrap-up

QLoRA = LoRA + quantisation

The frozen base is held in 4-bit, giving a large memory saving over plain LoRA; the small adapter carries the learning.

1. **4-bit frozen base** — stored in nf4 ("normal float", tuned for normally-distributed weights).
2. **De-quant to BF16** — on the fly, per layer, for the forward / backward pass.
3. **LoRA adapter trains** — only A and B updated.

- Trade-off: longer training and somewhat degraded quality.
- Newer 4-bit formats push further: MXFP4 lets gpt-oss run a 120B model on a single 80 GB GPU (a block of 32 values shares one scale).

Distillation is fine-tuning with generated data

- Same loop and loss as SFT — the only change is that **another model writes the data**.
- The "teacher" is a model you **rent** (API), not one you own or pre-train.
- Call it a few hundred times → input/output pairs, no human labelling.
- Fine-tune a small open student (Llama-8B, Qwen) on those pairs.
- Result: 5–30× cheaper inference at near-teacher quality on a narrow task.
- Caveat: check the teacher's terms of service before commercial use.

Key takeaways

1. **Fine-tune last, not first** — evals → prompting & context → fine-tune; fine-tune for a behaviour gap, not a knowledge gap.
2. **Beyond SFT** — SFT → DPO (preferences) → GRPO / RFT (reward-based, often verifiable). TRL ships all three.
3. **PEFT / LoRA is the substrate** — freeze the base, train $\Delta W = BA$; it underpins SFT, DPO and GRPO alike.
4. **Precision as a lever** — mixed precision (FP16/BF16) + quantisation (int8, fp8, nf4) → QLoRA fine-tunes billions of params on one GPU.

Hands-on lab (Topic4-unsloth.ipynb)

- Notebook LoRA-tunes a small instruct model (r = 16), base loaded in 4-bit.
- Change **one** hyperparameter and re-run: rank r, learning rate, epochs, or batch size / gradient accumulation steps.
- Remember: `max_seq_length` lives in `SFTConfig`, not `TrainingArguments`.

- [ ] Record trainable params (%), peak GPU memory, step time, final loss
- [ ] Keep before/after generations on the fixed prompts
- [ ] Bring results to the start of next class for comparison

Reality check: is fine-tuning worth it?

- Most teams never fine-tune, and that's fine — a good prompt against a strong API is the right default.
- The barrier is low: a few hundred clean examples change style, format or tone; a run costs ~$10 on one rented GPU.
- Classification is the exception that scales: budget per class, not per dataset.
- The gate is a behaviour gap, not budget. Fine-tuning has narrowed to a minority path for cost, latency or data residency.